# Chapitre 1: Les bases de la programmation en C

Arfaoui Ahlem

# 1) Introduction:

## 1.1 Historique:

* Le langage C a connu une croissance en popularité énorme ces dernières années. On trouve ses sources en 1972, dans les laboratoires Bell, afin de développer une version portable du système d'exploitation unix.
* C'est un langage de programmation structuré, mais très "près" de la machine.
* Publication en 1978 de "The C programming language" par Kernighan et Ritchie: définition classique du C.
* Le développement de compilateurs C par d'autres maisons ont rendu nécessaire la définition d'un standard précis: le standard ANSI-C.
* 1983: Développement par AT&T du C++.
* 1988: Seconde édition du livre « The C programming language ».
* 1990: Standard ANSI-C++.

## 1.2 Avantage:

**La portabilité:** Un programme développé en C sur une machine donnée peut être porté sur d'autres machines sans le modifier.
**Une grande bibliothèque de fonctions:** Le C, suivant les machines utilisées, dispose d'un grand nombres de fonctions, que ce soit des fonctions mathématiques, de gestion de fichiers ou d'entrées / sorties.

**Proche de la machine:** le C est très proche de la machine en pouvant accéder aux adresses des variables.
**Très rapide:** Aucun contrôle de débordement n'est effectué, ce qui apporte une plus grande vitesse.

## 1.2 Désavantages:

**Efficience et compréhensibilité :** en C, nous avons la possibilité d'utiliser des expressions compactes et efficientes. D'autre part, nos programmes doivent rester compréhensibles pour nous-mêmes et pour d'autres. Comme nous allons le constater sur les exemples suivants, ces deux exigences peuvent se contredire réciproquement.

**Exemple:**

Les deux lignes suivantes impriment les N premiers éléments d'un tableau A[], en insérant un espace entre les éléments et en commençant une nouvelle ligne après chaque dixième chiffre:

```
for (i=0; i<n; i++) printf("%6d%c", a[i], (i%10==9)?'\n':' ');
```

Cette notation est très pratique, mais plutôt intimidante pour un débutant. L'autre variante, plus près de la notation en Pascal, est plus lisible, mais elle ne profite pas des avantages du langage C:

```
for (I=0; I<N; I=I+1)
{ printf("%6d", A[I]);
 if ((I%10) == 9)
printf("\n");
 else
printf(" "); }
```

**=>**La premiere formulation de cette fonction est élégante, compacte, efficace et la traduction en langage machine fournit un code très rapide...; mais bien que cette manière de résoudre les problèmes soit le cas normal en C, il n'est pas si évident de suivre le raisonnement.

**Discipline de programmation**

**Les dangers de C:** Nous voici arrivés à un point crucial: C est un langage près de la machine, donc dangereux et bien que C soit un langage de programmation structuré, il ne nous force pas à adopter un certain style de programmation (comme p.ex. Pascal). Dans un certain sens, tout est permis et la tentation de programmer du 'code spaghetti' est grande. (Même la commande 'goto', si redoutée par les puristes ne manque pas en C). Le programmeur a donc beaucoup de libertés, mais aussi des responsabilités: il doit veiller lui-même à adopter un style de programmation propre, solide et compréhensible.

## 2) Structures d'un programme en C:

- **La fonction main:**

Elle constitue le programme principal:

```
Int main()
{
déclaration des variables
 instructions
}
```

- **Les fonctions**

**Type_du_resultat Nom_fonction (Type_param Nom_param,…)**
**{**
**déclaration des variables locales**
**instructions**
**}**

- **Les commentaires**

Un commentaire commence toujours par les deux symboles /* et se termine par les deux symboles*/.
Il est interdit d'utiliser des commentaires imbriqués.
**Exemple:** /* ceci est un commentaire correct */ /* ceci est /* évidemment */ incorrect */

- **Les variables**

En langage C, une variable est constituée de deux choses :
**1.Une valeur:** c'est le nombre qu'elle stocke, par exemple 5.
**2.Un nom:** c'est ce qui permet de la reconnaître.

**Syntaxe: type nom;**

- **Règles à respecter en C:**

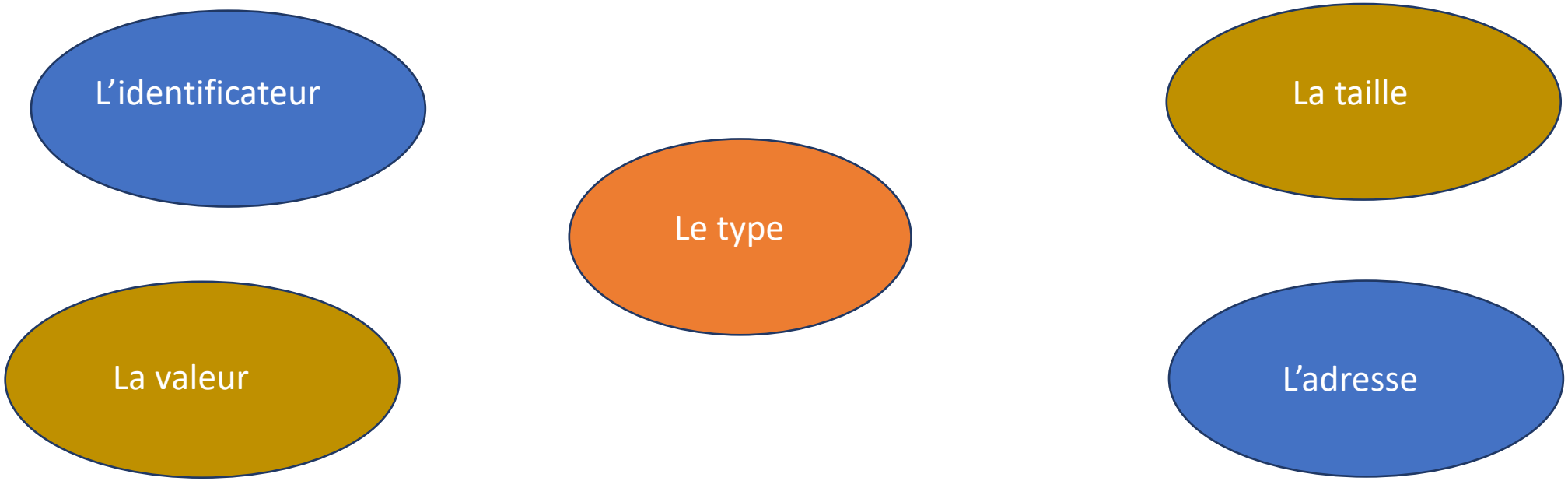
Les noms des fonctions et des variables en C sont composés d'une suite de lettres et de chiffres:

- le premier caractère doit être une lettre.
- L'ensemble des symboles utilisables est: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, …., x, y, z, A, B, C, …., X, Y, Z, _}
- Le C distingue les minuscules et les majuscules (Ma_Fonction # ma_fonction # MA_FONCTION)
- La longueur des identificateurs n'est pas limitée, mais le C distingue des 31 premiers caractères
- Un bloc d'instructions commence par **{** et se termine par **}**
- Il est déconseillé d'utiliser le symbole "_" comme premier caractère pour un identificateur, car il est souvent employé pour définir les variables globales de l'environnement C.
- les accents ("é", "à", "ê", etc) et les symboles (+, -, *, /, %, =) sont interdits.
- les espaces sont interdits mais on peut utiliser le caractère "underscore" _

## 3) Les variables et les constants:

**Le variable:** c'est une petite information temporaire qu'on stocke dans la mémoire vive d'une machine.
**Le constant:** c'est une information qu'on stocke dans la mémoire morte d'une machine.
**Remarque:**
Par convention, on écrit les noms des constantes tout en **majuscules**. Cela permet de distinguer facilement les constantes des variables.

Chaque variable et constant est défini par:



## 4) Les types

**- Les entiers**

| Définition | Description | Valeur min | Valeur max | Nombre d'octets |
|---|---|---|---|---|
| char | caractère | -128 | 127 | 1 |
| short | Entier court | -32768 | 32767 | 2 |
| int | Entier standard | - 32768 | 32767 | 4 |
| long | Entier long | -2147483648 | 214748364è | |

Si l'on ajoute le préfixe unsigned (non signé), les domaines sont déplacés ainsi:

| Définition | Valeur min | Valeur max |
|---|---|---|
| Unsigned char | 0 | 255 |
| Unsigned short | 0 | 65535 |
| Unsigned int | 0 | 65535 |
| Unsigned long | 0 | 4294967295 |

**-les flottants:**

| Définition | Description | Nombre d'octets |
|---|---|---|
| float | simple | 4 |
| double | double | 8 |
| Long double | avancée | 10 |

**-Le booléen**:

il ne peut prendre que deux états, VRAI ou FAUX

## 5) Les bibliothèques de fonctions prédéfinies

### *Utilisation des bibliothèques de fonctions*

La pratique en C exige l'utilisation de bibliothèques de fonctions. Ces bibliothèques sont disponibles dans leur forme précompilée (extension: .LIB). Pour pouvoir les utiliser, il faut inclure des fichiers en-tête (header files - extension .H) dans nos programmes. Ces fichiers contiennent des 'prototypes' des fonctions définies dans les bibliothèques et créent un lien entre les fonctions précompilées et nos programmes.

**#include**

L'instruction **#include** insère les fichiers en-tête indiqués comme arguments dans le texte du programme au moment de la compilation.

Identification des fichiers

Lors de la programmation en Borland C, nous travaillons donc avec différents types de fichiers qui sont identifiés par leurs extensions:

***.C:** fichiers source

***.OBJ:** fichiers compilés (versions objet)

***.EXE:** fichiers compilés et liés (versions exécutables)

***.LIB:** bibliothèques de fonctions précompilées

***.H:** fichiers en-tête (header files)

**Exemple**

Nous avons écrit un programme qui fait appel à des fonctions mathématiques et des fonctions graphiques prédéfinies. Pour pouvoir utiliser ces fonctions, le programme a besoin des bibliothèques:

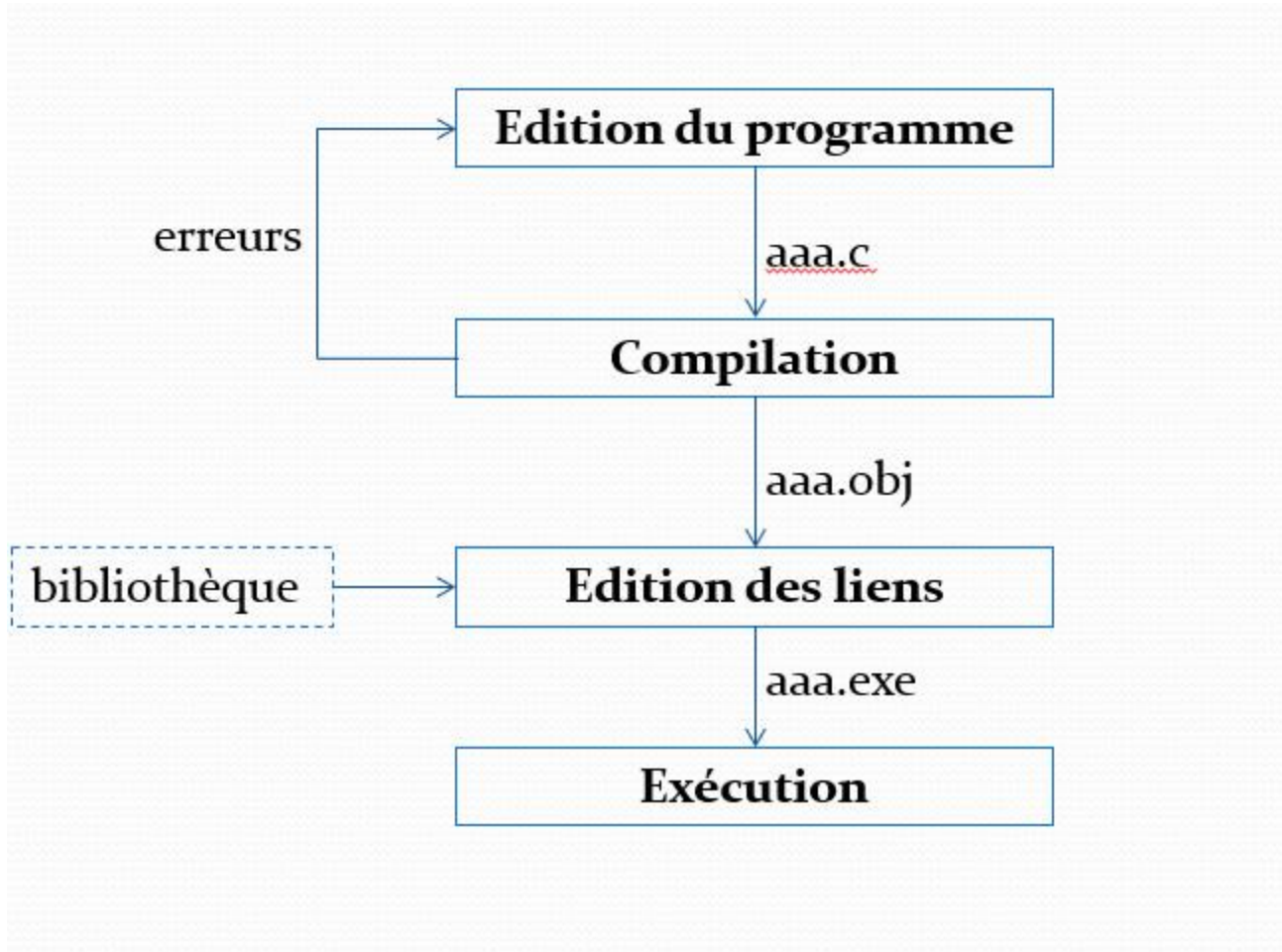
**MATHS.LIB**

**GRAPHICS.LIB**

Nous devons donc inclure les fichiers en-tête correspondants dans le code source de notre programme à l'aide des instructions:

```
#include <math.h>
 #include <graphics.h>
```

Après la compilation, les fonctions precompilées des bibliothèques seront ajoutées à notre programme pour former une version exécutable du programme .


Edition du programme
erreurs
aaa.c
Compilation
aaa.obj
bibliothèque
Edition des liens
aaa.exe
Exécution

**Exemple de programme:**

```
#include <stdio.h>
#include <stdlib.h>  //Ils contiennent du code tout prêt qui permet d'afficher du texte à l'écran.
int main()
{
   printf("Hello world!\n");  //demande à afficher le message "Hello world!" à l'écran
   return 0;
}
```

=> En principe tout programme devrait retourner une valeur comme code d'erreur à son environnement. Par conséquent, le type résultat de main est toujours int. En général, le type de main n'est pas déclaré explicitement, puisque c'est le type par défaut. Nous allons terminer nos programmes par l'instruction:

return 0;

qui indique à l'environnement que le programme s'est terminé avec succès, sans anomalies ou erreurs fatales.

## 6) Les opérateurs standards:

- **L'affectation (=):**

**Nom_variable=expression;**

Affectation avec des valeurs constantes:

```
Pi = 3.1416;
Lettre = 'L';
```

Affectation avec des valeurs variables: a=b;

- **Les opérateurs arithmétiques**

+ addition

- soustraction

*multiplication

/ division

% modulo (reste de la division entière) (par exemple, 5%2=1)

- **Les opérateurs logiques**

&& ET

|| OU

! NON

Les résultats des opérations logiques sont de type int: la valeur 0 correspond à la valeur booléenne FAUX, la valeur 1 correspond à la valeur booléenne VRAI.

- **Les opérateurs de comparaison**

== EGALITE

!= INEGALITE

< INFERIEUR

<= INFERIEUR OU EGAL

>SUPERIEUR

>= SUPERIEUR OU EGAL

⇒ Les résultats des opérations de comparaison sont de type int: la valeur 0 correspond à la valeur booléenne FAUX, la valeur 1 correspond à la valeur booléenne VRAI Les opérateurs logiques considèrent toute valeur différente de 0 comme VRAI, toute valeur nulle comme FAUX

- **L'opérateur d'affectation**

i = i + 2;

En C, on peut écrire:

i+ = 2;

Pour la plupart des expressions de la forme: expr1 = (expr1) Opérateur (expr2); Il existe une formulation équivalente:expr1 Opérateur = expr2; Cette formulation (i+ = 2) suit la logique humaine: on ajoute 2 à i. L'opérateur d'affectation aide le compilateur à générer un code plus efficace car expr1 n'est évalué qu'une fois.

- **L'opérateur d'incrémentation et de décrémentation**

i = i + 1 s'écrit: i++ ou ++i

i = i - 1 s'écrit: i-- ou –i

**Les opérateurs ++ et --** sont utilisés:

- Pour incrémenter ou décrémenter une variable, par exemple dans une boucle (dans ce cas, pas de différence entre la notation préfixe (++i,--i) et la notation postfixe (i++,i--).
- Pour incrémenter ou décrémenter une variable et en même temps affecter sa valeur à une autre variable. Dans ce cas, on choisit entre la notation préfixe et postfixe:
- X = i++; passe d'abord la valeur de i à X, puis incrémente i (le ++ est après i, on l'incrémente après) X = i--; passe d'abord la valeur de i à X, puis décrémente i
- X = ++i; incrémente d'abord i puis passe la valeur de i incrémentée à X (le ++ est avant i, on l'incrémente avant) X = --i; décrémente d'abord i puis passe la valeur de i décrémentée à X

**Exemple 1:**

N = 5;

X = N++;

**Résultat:**

X = 5 et N = 6

**Exemple 2:**

N = 5;

X = ++N;

**Résultat:**

X = 6 et N = 6

- **Priorités des opérateurs**

L'ordre d'évaluation des différentes parties d'une expression correspond à celle que nous connaissons en maths. Exemple:

A = 3;

B = 4;

X = 2 * A + 3 * B;

6 + 12

18

Ensuite, on affecte 18 à X

Comme en maths, si l'on veut forcer l'ordinateur à commencer par un opérateur de priorité plus faible, on entoure le terme par des parenthèses.
Exemple:

```
A = 3;
B = 4;
 X = 2 *(A + 3) * B;
       2 * 6
           12 * 4
                 48
```

**Classes de priorités:**

Priorité 1 (la plus élevée) ( )
Priorité 2 !, ++, --
Priorité 3 *, /, %
Priorité 4 +, -
Priorité 5 <=, >, >=
Priorité 6 ==, !=
Priorité 7 &&
Priorité 8 ||
Priorité 9 (la plus basse) =, +=, -=, *=, /=, %=

=> Dans chaque classe, les opérateurs ont la même priorité. Si on a une suite d'opérateurs binaires de la même classe, l'évaluation se fait en passant de la gauche vers la droite. : pour les opérateurs unaires (!, ++, --) et pour les opérateurs d'affectation, l'évaluation se fait de la droite vers la gauche

## 7) Lire et écrire des données

La bibliothèque standard **<studio.h>** contient un ensemble de fonctions qui assurent la communication de la machine avec le monde extérieur.

Les fonctions les plus importantes sont:

**Pour l'écriture:**

printf(): écriture formatée de données

Getchar: écriture d'un caractère

**Pour lecture:**

scanf(): lecture formatée de données

putchar(): lecture d'un caractère

### 7.1 Ecriture formatée de données

**printf():** cette fonction est utilisée pour transférer du texte, des valeurs de variables ou des résultats d'expression vers le fichier de sortie standard stdout (par défaut l'écran).

**Syntaxe: printf("format", expr_1, expr_2)**

"format" est une chaîne de caractère qui peut contenir:

- du texte
- des séquences d'échappement
- des spécificateurs de format (un spécificateur pour chaque expression)

**Exemple**

```
printf("La somme est: %i \n", SOMME);
```

Les spécificateurs de format: ils commencent toujours par le symbole %

| Symbole | Impression comme | Type |
|---|---|---|
| %d ou %i | Entier relatif | int |
| %u | Entier naturel (non signé) | int |
| %o | Entier exprimé en octal | int |
| %x | Entier exprimé en hexadécimal | int |
| %f | Rationnel en notation décimale | float |
| %e | Rationnel en notation scientifique | float |
| %g | Rationnel en notation décimale/scientifique | float |
| %lf | Rationnel en notation décimale | double |
| %lg | Rationnel en notation décimale/scientifique | double |
| %le | Rationnel en notation scientifique | double |
| %c | Caractère | char |
| %s | Chaîne de caractère | char* |

Arguments de type long:

Entiers: on utilise: %ld, %li, %lu, %lo, %lx

Rationnels: pour les "long double", on utilise: %Lf, %Lg ou %Le

**7.2 Ecriture formatée de données**

scanf(): fonction symétrique de printf().

Syn**taxe: scanf("format", adr_var_1, adr_var_2)**

- La fonction scanf reçoit ses données à partir du fichier standard stdin (le clavier)
- La chaîne de format détermine comment les données reçues doivent être interprétées
- Les données reçues correctement sont mémorisées aux adresses indiquées par adr_var_1, adr_var_2, …

Les spécificateurs de format: ils commencent toujours par le symbole %

| Symbole | Impression comme | Type |
|---|---|---|
| %d ou %i | Entier relatif | int |
| %u | Entier naturel (non signé) | int |
| %o | Entier exprimé en octal | int |
| %x | Entier exprimé en hexadécimal | int |
| %f | Rationnel en notation décimale | float |
| %e | Rationnel en notation scientifique | float |
| %g | Rationnel en notation décimale/scientifique | float |
| %lf | Rationnel en notation décimale | double |
| %lg | Rationnel en notation décimale/scientifique | double |
| %le | Rationnel en notation scientifique | double |
| %c | Caractère | char |
| %s | Chaîne de caractère | char* |

**7.3 Ecriture d'un caractère**

**Syntaxe: putchar (caractere);**

putchar transfère le caractère "caractere" vers le fichier de sortie standard stdout (l'écran), les arguments de putchar sont des caractères (type char, i.e. des nombres entiers entre 0 et 255).

**Exemples:**

Affichage putchar('x’);  x

putchar('?’);  ?

putchar(65); A (65 est le code ASCII de A)

putchar('\n'); retour à la ligne

putchar(A); valeur de la variable A si c'est un char

**7.4 Lecture d'un caractère**

**Syntaxe: getchar ();**

Les valeurs retournées par getchar() sont des caractères. Le type du résultat de getchar est int.

**Exemple:**

int C;

C=getchar();

getchar lit les données de la zone tampon stdin (clavier) et fournit les données seulement après confirmation par "Enter".